CS4475 Fall 2026
Emmanuel Barraza
ebarrazavi@miners.uep.edu

Lab Report
Submitted on 10/14/26

PA2: Priority-based Scheduler for xv6

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.
(pstat.h , user.h , usys.pl, syscall.h, syscall.c, defs.h, sysproc.c, proc.c, proc.h)
Are all the files that were changed to implement the getprocs() system call, which I implemented by first defining the "struct pstat" in pstat.h and adding the "int getprocs(struct pstat*);" prototype into user.h. I next implemented or registered "getprocs" across the xv6 in the users and kernel files as a new system call. At the end I implemented getprocs() in proc.c that iterates through the process table which collects information and transfers it into user provided array and also returns count of running processes found.

```
xv6 kernel is booting

init: starting sh
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
3       running         16384   2       ps
$
```

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.
To modification i had to make to getprocs(), was by implementing the process priority line so it can copy each process's priority out of kernel so the ps in user can see and print it.

```
init: starting sh
$ testprio
Default priority: 0
Updated priority: 15
pid     state           size    ppid    pri     name
1       sleeping        16384   0       0       init
2       sleeping        20480   1       0       sh
3       sleeping        16384   2       15      testprio
4       running         16384   3       15      ps
$
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

The way the process's age is calculated is by current time minus readytime, or how long it been waiting for its turn on the CPU. Which the readytime starts when process transition to RUNNABLE and ends or age is zero when in another state like RUNNING or SLEEPING

```
xv6 kernel is booting

init: starting sh
$ ps
pid     state           size    ppid    pri     age     name
1       sleeping        16384   0       0       0       init
2       sleeping        20480   1       0       0       sh
3       running         16384   2       0       0       ps
$
```

Task 4. (15 pts) Implement a priority-based scheduler.

Param.h : added constants to file so it can choose between scheduling policies at compile time

Proc.c : implemented the priority-based scheduler in the scheduler function and preserved the default round-robin under a "#else". The priority-based scheduler was implemented to select the highest priority process to run next.

What I learned about carrying out this task is having a better understanding about how a priority scheduler works like. In that by implementing a priority-based scheduler as it ensures the most important task is always jumped to the front.

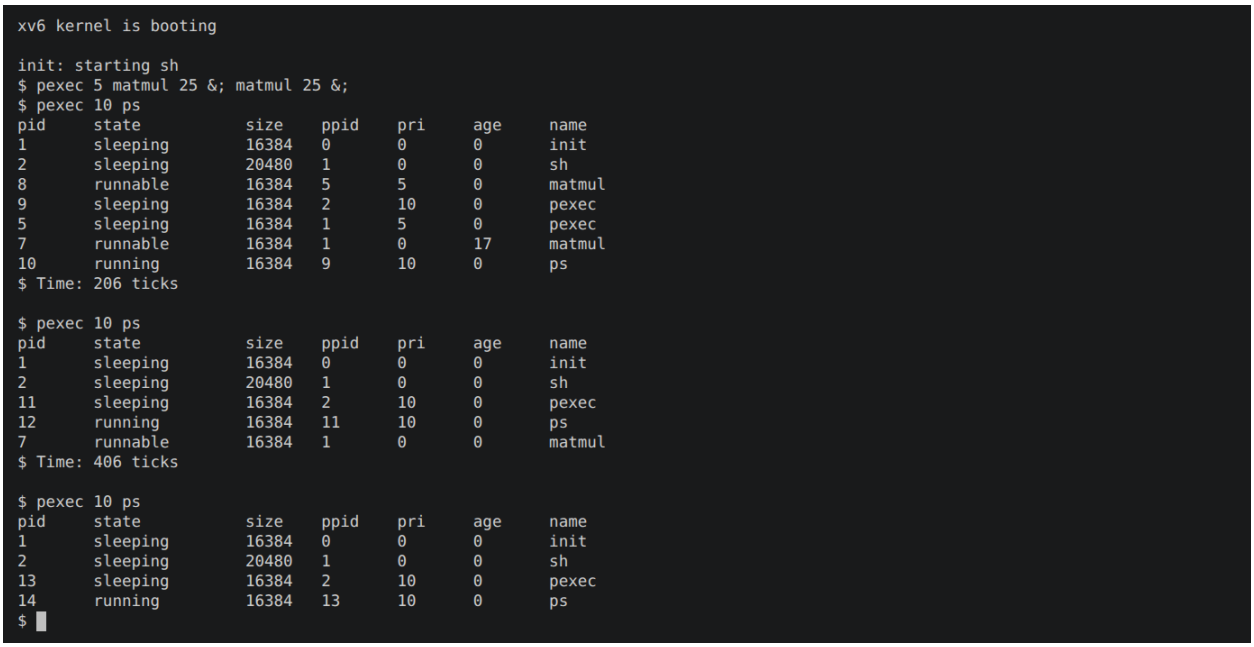
The main difficulty I faced with this task was implementing the priority-based scheduler in the scheduler function on the proc.c file. The way i overcame this was by looking through the existing round-robin in the scheduler function, which helped me implement the priority-based scheduler.

```
xv6 kernel is booting

init: starting sh
$ priotest
Child 2 (HIGH 40) started
Child 2 (HIGH 40) done
Child 1 (LOW 10) started
Child 1 (LOW 10) done
$
```

Task 5. (15 pts) Add aging to your priority-based scheduler.

The aging implemented in my priority-based scheduler works by making the process a higher priority if it waits for too long. The code follows most of the priority-based scheduler, but with aging feature in which age is calculated by "ticks - readytime", and if it waits over 10 timer ticks its priority increases prevent starvation.
Carrying out this task I was able to learn how effective aging is, by preventing starvation since it makes sure urgent processes are run first. Within the lab one of the difficulties I run into was implementing the aging, the way i overcame this was by going through how aging works and implementing it into code in the priority-based scheduler.



Task 6. (5 pts) Summary

Through this assignment i learned how a system call works to be able to search in kernels process list and copy those details into user to be able to display running process. How to store the priority into each process kernel record in allocproc() and use the systems calls so user can read and change the kernel property safely. Also, i was able to learn how to track the waiting time for process when it transitions from one state to RUNNABLE, by calculating ticks – readytime to get age. The main part learned in this assignment was about implementing the priority-based scheduler in that I learn how to CPU scheduling works like but causes starvation in that low-priority tasks never run, so an aging policy was implemented to prevent starvation by making process that wait too long a higher priority.

The main obstacle I faced was implementing the priority-based scheduler and its aging policy within task 4-5. The way I was able to face this obstacle was when implementing the priority-based scheduler was by looking at the round-robin policy code and what can be changed or added to implement the priority-based scheduler. As for the aging policy, I went through how aging works to prevent starvation, how it can be implemented to the priority-based scheduler, and which if a process waits for too long for a certain time its priority increases.

Extra Credit Task (10 points).

The way I implemented getpriority and setpriority commands, is by in the getpriority I added an integer priority, and made it so every new process start with a default priority of 0 in allocproc(). I also set fork() so that child processes inherit parents' priority. So this command finds the currently running process. Within setpriority, i implemented a system call that accepts new priority number from user and checks if its within the valid ranges of 0 – 49. If valid it, it safely locks the process, replaces the old priority with the new value, unlocks the process, and returns the success code or 0.

Using these commands can improve response time for interactive jobs, by how setpriority can give interactive jobs a higher priority letting it hump straight into the front of ready queue instead of having to wait instead of starving. As for getpriority it helps interactive jobs by letting them check current priority before deciding to change is needed.